SARVAA Fresh
Software Architecture Document (ARCHITECTURE.md)
Version: 2.0
Prepared By: Principal Software Architect
Date: June 2026

---

1. Executive Summary

SARVAA Fresh is being designed as a modern hyperlocal commerce platform starting with fruits and vegetables and expanding into groceries, dairy, household products, local vendors, subscriptions, and future commerce services.
The architecture prioritizes:

- **Fast MVP development**
- **Easy future modifications**
- **Long-term scalability**
- **Developer productivity**
- **AI-assisted development**
- **Large ecosystem support**
- **Easy hiring and onboarding**
- **Smooth migration to microservices**

The MVP will launch as a Modular Monolith with clearly separated business domains.
The technology stack is selected based on long-term maintainability and ecosystem maturity rather than personal familiarity.

---

2. Architectural Principles

Core Principles

API First

All functionality must be exposed through APIs.
Benefits:

- **Mobile app readiness**
- **Third-party integrations**
- **Easy frontend replacement**
- **Future marketplace integrations**

Domain Driven Structure

Modules organized by business capability.
Example:
Good
Catalog Module
Order Module
Inventory Module
Payment Module
Bad
Controllers
Services
Repositories
Business domains should remain independent.

Category Agnostic Design

Never build:

- **Fruit Module**
- **Vegetable Module**

Build:

- **Product Module**

Future categories must be configurable through data, not code.

Modular Monolith First

MVP:

Single Backend Application

Future:

Independent services when required

Benefits:

- **Faster development**
- **Easier debugging**
- **Lower operational complexity**
- **Easier AI-assisted development**

Scalability by Design

Every module should be designed so it can later become an independent service without major rewrites.

Examples:

- **Auth Service**
- **Catalog Service**
- **Inventory Service**
- **Order Service**
- **Payment Service**

AI-Friendly Development

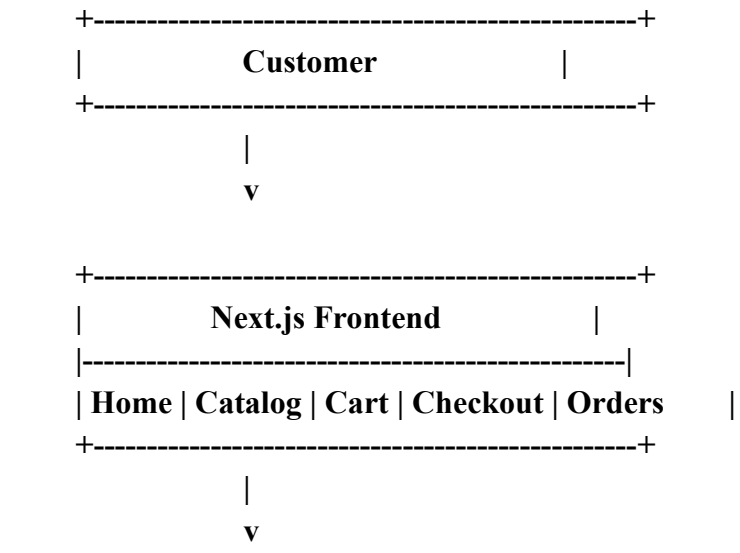
Every module should have:

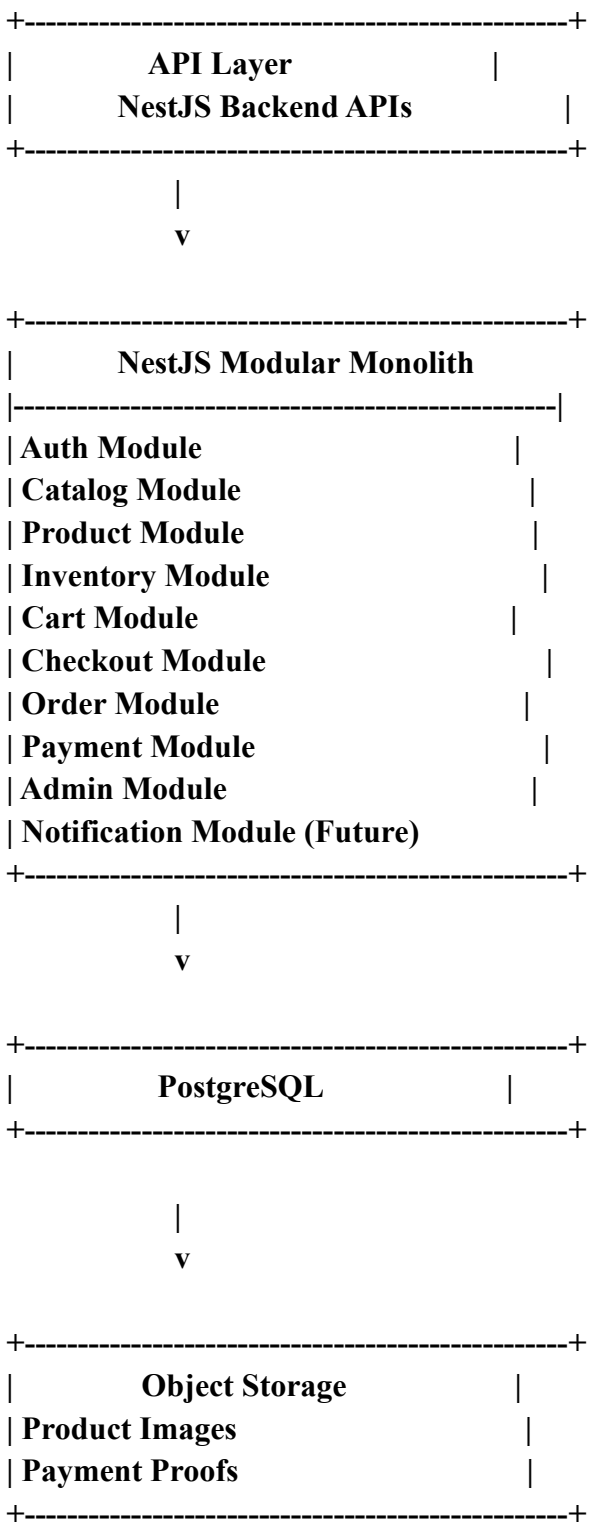
- **Clear boundaries**
- **Independent documentation**
- **API contracts**
- **Database ownership rules**

This allows AI agents and developers to work independently.

---

3. High-Level Architecture Diagram





---

4. System Modules

Customer Domain

Authentication Module

Responsibilities:

- **Registration**
- **Login**
- **Session Management**
- **Password Reset**

Future:

- **OTP Login**
- **Google Login**
- **Apple Login**

- **Social Authentication**

Catalog Module
Responsibilities:
- **Category Listing**
- **Product Discovery**
- **Search**
- **Filtering**
- **Recommendations**

Product Module
Responsibilities:
- **Product Details**
- **Product Variants**
- **Product Images**
- **Pricing**
- **Product Metadata**

Cart Module
Responsibilities:
- **Add To Cart**
- **Remove Items**
- **Quantity Management**
- **Cart Persistence**

Checkout Module
Responsibilities:
- **Address Selection**
- **Delivery Scheduling**
- **Payment Selection**
- **Order Validation**

Order Module
Responsibilities:
- **Order Creation**
- **Status Tracking**
- **Order History**
- **Order Lifecycle Management**

Payment Module
Responsibilities:
- **COD Orders**
- **UPI Verification**
- **Payment Tracking**

Future:
- **Razorpay**
- **PhonePe**
- **Paytm**
- **Stripe**

Admin Domain
Dashboard Module
Responsibilities:

- **Revenue Metrics**
- **Orders Overview**
- **Inventory Alerts**
- **Business Analytics**

Inventory Module
Responsibilities:

- **Stock Updates**
- **Stock Reservation**
- **Low Stock Detection**
- **Inventory Auditing**

Product Management Module
Responsibilities:

- **Product CRUD**
- **Variant Management**
- **Image Management**
- **Category Management**

Order Management Module
Responsibilities:

- **Status Updates**
- **Order Assignment**
- **Delivery Monitoring**
- **Refund Handling**

---

5. Frontend Architecture
Technology:

- **Next.js 15+**
- **TypeScript**
- **TailwindCSS**
- **ShadCN UI**
- **React Query**

Reason:

- **Large ecosystem**
- **Excellent developer experience**
- **Easy maintenance**
- **Strong AI tooling support**
- **Enterprise scalability**

Folder Structure
src/

├── app/
│

```
├── components/
│
├── modules/
│   ├── auth/
│   ├── catalog/
│   ├── cart/
│   ├── checkout/
│   ├── orders/
│   └── admin/
│
├── services/
│
├── hooks/
│
├── store/
│
├── lib/
│
├── providers/
│
└── types/
```

Frontend Pages
Customer
/
/catalog
/product/[slug]
/cart
/checkout
/orders
/profile
/login
/register

Admin
/admin
/admin/products
/admin/categories
/admin/inventory
/admin/orders
/admin/customers

State Management
MVP:

- **React Query**
- **Server Components**

Future:

- **Zustand**

Reason:

Simple, scalable, and widely adopted.

---

6. Backend Architecture
Technology:

- **NestJS**
- **TypeScript**
- **REST APIs**

Architecture Style:
Modular Monolith
Reason:

- **Faster development**
- **Easier onboarding**
- **Huge ecosystem**
- **Enterprise-ready architecture**
- **Easier future modifications**
- **Strong AI coding support**
- **Easier migration to microservices**

Backend Folder Structure
src/

```
├── modules/
│   ├── auth/
│   ├── catalog/
│   ├── products/
│   ├── inventory/
│   ├── cart/
│   ├── checkout/
│   ├── orders/
│   ├── payments/
│   └── admin/
│
├── common/
│
├── config/
│
├── database/
│
└── shared/
```

Layer Structure
Each Module:
module/

controllers
services
entities
dto
repositories

guards
validators
Example:
orders/

controllers
services
entities
dto
repositories

API Design
REST API
Example:
/api/v1/auth

/api/v1/products

/api/v1/orders

/api/v1/inventory
Versioning from Day 1.
Future:
GraphQL can be added without major changes.

---

7. Database Architecture
Technology:
PostgreSQL
ORM:
Prisma ORM
Reason:

- **Excellent developer experience**
- **Easy schema management**
- **Type safety**
- **Faster development**
- **Easier future modifications**

Core Tables
users

addresses

categories

products

product_variants

inventory

carts

cart_items

orders

order_items

payments

admins

order_status_history

Future Tables
vendors

notifications

coupons

reviews

wishlists

subscriptions

loyalty_points

delivery_partners

deliveries

Database Principles
Soft Delete
Use:
deleted_at
instead of hard delete.

Audit Fields
Every table:
created_at

updated_at

UUID Based IDs
Use:
uuid

Benefits:

- **Better security**
- **Easier distributed systems migration**
- **Easier microservice adoption**

---

8. File Storage Architecture
MVP
Use:
AWS S3
or
Cloudflare R2
Stores:

- **Product Images**
- **Payment Proofs**
- **Future Documents**

Reason:
Avoid future migration from local storage.

---

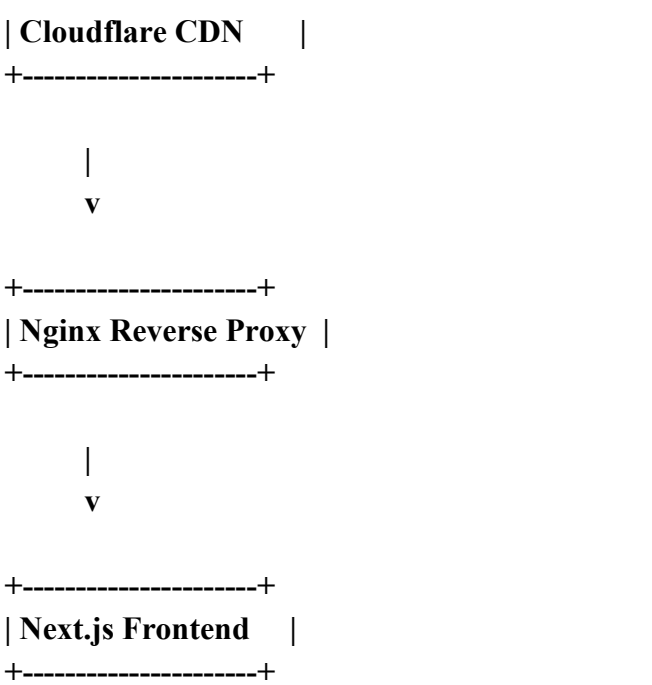
9. Deployment Architecture
MVP Deployment
Docker Compose
Services:
Frontend

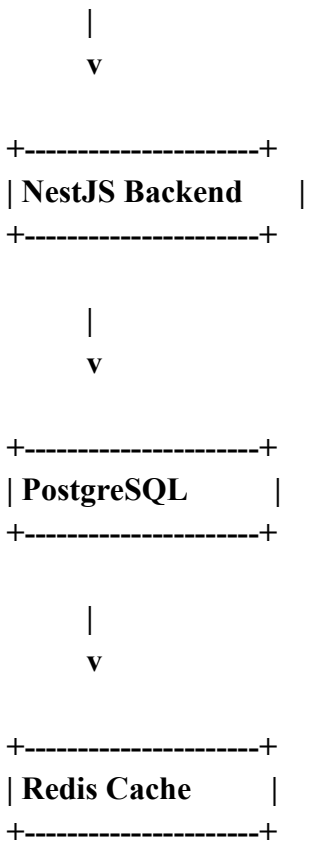
Backend

PostgreSQL

Redis

Production Layout





Environment Management
Separate:
development

staging

production
Environment Variables:
DATABASE_URL

JWT_SECRET

REDIS_URL

AWS_ACCESS_KEY

AWS_SECRET_KEY

AWS_BUCKET_NAME

---

10. Security Considerations
Authentication
JWT Authentication
Future:

- **Refresh Tokens**
- **Device Sessions**
- **OAuth Providers**

Password Security
Use:
Argon2
hashing
Never store plain passwords.

API Security

- **Input validation**
- **Rate limiting**
- **Request logging**
- **Security headers**

File Upload Security
Validate:

- **File Type**
- **File Size**
- **Malware Checks**

Database Security
Least Privilege Principle
Separate:

- **Application User**
- **Admin User**

HTTPS
Mandatory in production.

---

11. Observability
MVP

- **Structured Logs**
- **Error Logs**
- **Access Logs**

Future

- **Prometheus**
- **Grafana**
- **Loki**
- **OpenTelemetry**
- **Sentry**

---

12. Scalability Roadmap
Stage 1
0–5,000 Users
Architecture:

- **Single Server**
- **Docker Compose**

Stage 2
5,000–50,000 Users

Add:

- **Redis Cache**
- **CDN**
- **Queue System**
- **Object Storage**

Stage 3
50,000–500,000 Users
Add:

- **Read Replicas**
- **Background Workers**
- **Dedicated Search Service**

Stage 4
500,000+ Users
Move:

- **Kubernetes**
- **Auto Scaling**
- **Distributed Services**
- **Multi-Region Deployment**

---

13. Future Microservice Migration Strategy
The MVP remains a Modular Monolith.
Do NOT start with microservices.

Extraction Order
Service 1
Notification Service
Reason:
Independent workload

Service 2
Payment Service
Reason:
External integrations

Service 3
Inventory Service
Reason:
High write volume

Service 4
Order Service
Reason:
Business critical

Service 5
Catalog Service
Reason:

High read traffic

Target Architecture
Frontend

|

API Gateway

|

Auth Service

Catalog Service

Inventory Service

Order Service

Payment Service

Notification Service

PostgreSQL

Redis

Kafka

Object Storage

---

14. AI-Agent Development Strategy
Architecture Agent
Maintains:

- **System Design**
- **Documentation**
- **Technical Decisions**

Backend Agent
Owns:

- **NestJS APIs**
- **Database Integration**
- **Business Logic**

Frontend Agent

Owns:

- **Next.js UI**
- **User Experience**
- **State Management**

QA Agent
Owns:

- **Testing**
- **Validation**
- **Automation**

DevOps Agent
Owns:

- **Docker**
- **CI/CD**
- **Infrastructure**
- **Monitoring**

Final Architecture Decision
For the first launch, SARVAA Fresh will use:
Frontend:

- **Next.js**
- **TypeScript**
- **TailwindCSS**
- **ShadCN UI**
- **React Query**

Backend:

- **NestJS**
- **TypeScript**
- **Modular Monolith Architecture**

Database:

- **PostgreSQL**
- **Prisma ORM**

Caching:

- **Redis**

Storage:

- **AWS S3 or Cloudflare R2**

Infrastructure:

- **Docker Compose**

This architecture is optimized for rapid development, easy future modifications, strong AI-assisted development, large developer ecosystem support, and a smooth path from MVP to enterprise-scale hyperlocal commerce without major rewrites.